

University of Information Technology and Sciences (UITS)

Department of Computer Science and Engineering

Compiler LAB • Course Code: CSE 352

PROJECT PROPOSAL

NetXLang

A Network-Oriented Bengali Domain-Specific Language

(Bengali-Inspired Networking & Security DSL)

A Domain-Specific Language Implemented with a C++ Compiler

Student Name	Sabikun Nahar Alina
Student ID	0432320005101016
Batch	CSE 54
Course	Compiler LAB
Course Code	CSE 352
Language Name	NetXLang
File Extension	.nxi
Assigned Date	April 13, 2025
Due Date	April 25, 2025
Total Marks	10 Points
Submitted To	Md. Tasnin Tanvir, Dept. of CSE, UITS

1. Introduction

NetXLang is a custom domain-specific language (DSL) designed for network simulation and security modeling. It abstracts real-world networking operations packet transmission, routing, firewall enforcement, encryption, and traffic monitoring into simple, Bengali-inspired function keywords. Engineers and students can write readable network simulation scripts without deep knowledge of socket programming or low-level protocol APIs. NetXLang source files use the `nxl` extension and are processed by a C++17 compiler pipeline.

2. Motivation & Objectives

Network engineering concepts are typically taught through heavy protocol documentation and complex socket code. NetXLang bridges this gap by providing an expressive, human-readable scripting layer where every keyword reflects a real networking operation. The objectives are:

- Design a domain-focused DSL where every keyword maps to a real network concept no generic renamed print/if pairs.
- Implement a full compiler pipeline (Lexer → Parser → Semantic Analyzer → Interpreter) in C++17.
- Cover 25 unique networking primitives: device management, packet flow, routing, security, and monitoring.
- Use Bengali-inspired vocabulary to make the language accessible to Bangladeshi CSE/IT students.
- Simulate core networking scenarios topology setup, packet transmission, firewall rules, and encryption.

3. Language Overview

Language Name: NetXLang | **File Extension:** `.nxl` | **Compiler:** C++17

Sample NetXLang Program:

```
// NetXLang sample – simple encrypted packet simulation
NetArambho
JontraGothon("ServerA") JontraGothon("ClientB") ThikanaDao("ServerA",
"192.168.1.1") ThikanaDao("ClientB", "192.168.1.2") DorjaDao("ServerA", 8080)
JogajogSet("ServerA", "ClientB") NiyomBoshao("ServerA", "TCP")
ProtirodhDao("ServerA", "BLOCK", "192.168.1.99") GoponKoro("SecretMessage")
PacketPathao("ClientB", encrypted_data) PacketNey("ServerA")
UghatKoro(received_data) NirikhaKoro("ServerA") GhotonaLekho("Packet received and
decrypted") Dekhao "Transfer complete" NetShesh
```

4. Compiler Architecture (C++)

- **Lexer (Tokenizer):** Reads `.nxl` source files and produces tokens NetXLang keywords, device identifiers, IP strings, port numbers, and string literals.
- **Parser:** Builds an Abstract Syntax Tree (AST) from the token stream using NetXLang grammar rules including device declarations and packet flow statements.
- **Semantic Analyzer:** Validates device existence before connection, checks IP/port format, confirms function signatures and routing path consistency.
- **Interpreter / Simulator:** Traverses the AST and simulates network operations maintaining a virtual device registry, link map, and packet queue.

Tools & Technologies:

- Language: C++17 | Build System: g++ / Makefile
- Crypto Simulation: OpenSSL (AES-256 for GoponKoro / UghatKoro)
- Platform: Linux / Windows (cross-platform)

5. Function Table of NetXLang

The table below lists all **25 unique functions** of NetXLang. Each keyword is domain-specific every function reflects a real networking operation with no generic aliases.

Standard Concept	NetXLang Function	Short Description
Start Network	NetArambho ()	Initialize the network environment
End Network	NetShesh ()	Shut down the network session
Create Device	JontraGothon ()	Create a virtual node or device
Connect Devices	JogajogSet ()	Establish a link between two nodes
Assign IP	ThikanaDao ()	Assign an IP address to a device
Assign Port	DorjaDao ()	Assign a communication port
Send Packet	PacketPathao ()	Transmit a data packet to a target
Receive Packet	PacketNey ()	Accept and buffer incoming data
Broadcast	ShobaiPathao ()	Send a packet to all connected nodes
Routing	PothNirdharon ()	Define a routing path between nodes
Ping	JachaiPing ()	Check reachability of a device
Delay	Bilombho ()	Simulate network latency in ms
Drop Packet	HariyeFelo ()	Simulate packet loss on a link
Bandwidth Control	GotiNiyontron ()	Set maximum data throughput
Set Protocol	NiyomBoshao ()	Define protocol (TCP/UDP/HTTP)
Firewall Rule	ProtirodhDao ()	Apply a security/firewall rule
Encrypt	GoponKoro ()	Encrypt a data payload
Decrypt	UghatKoro ()	Decrypt an encrypted payload
Monitor Traffic	NirikhaKoro ()	Start traffic monitoring on a node
Log Event	GhotonaLekho ()	Write an event to the network log
Condition	Jodi ()	Conditional if-block for logic
Else	Nahole ()	Handles the alternative condition
Loop	GhuroChol ()	Repeat a block a fixed number of times
Function Define	KajGothon ()	Define a reusable network function
Output	Dekhao ()	Print output to the console/log

6. Implementation Plan

Phase	Task	Deliverable
1	Design grammar rules and NetXLang keyword vocabulary	Grammar spec (.bnf file)

Phase	Task	Deliverable
2	Implement Lexer in C++ – tokenize .nxl source files	lexer.cpp / lexer.h
3	Implement Parser and build Abstract Syntax Tree (AST)	parser.cpp / ast.h
4	Semantic analysis, scope resolution, type checking	semantic.cpp
5	Interpreter with simulated network backend for 25 functions	interpreter.cpp
6	Testing all functions with sample .nxl programs	Test cases + final demo

7. Conclusion

NetXLang is a well-scoped and technically feasible project for the Compiler LAB course. Unlike generic renamed-keyword languages, every one of NetXLang's 25 functions reflects a real networking concept from packet transmission and routing to firewall enforcement and traffic encryption. The Bengali-inspired naming makes the language accessible to Bangladeshi students while the full C++ compiler pipeline (Lexer, Parser, Semantic Analyzer, Interpreter) covers all core aspects of the course. The final submission will include a working compiler, sample .nxl programs demonstrating each of the 25 functions, and a comprehensive test suite.

Submitted By: ALINA | ID: 1016 | Batch: CSE 54